DentTime

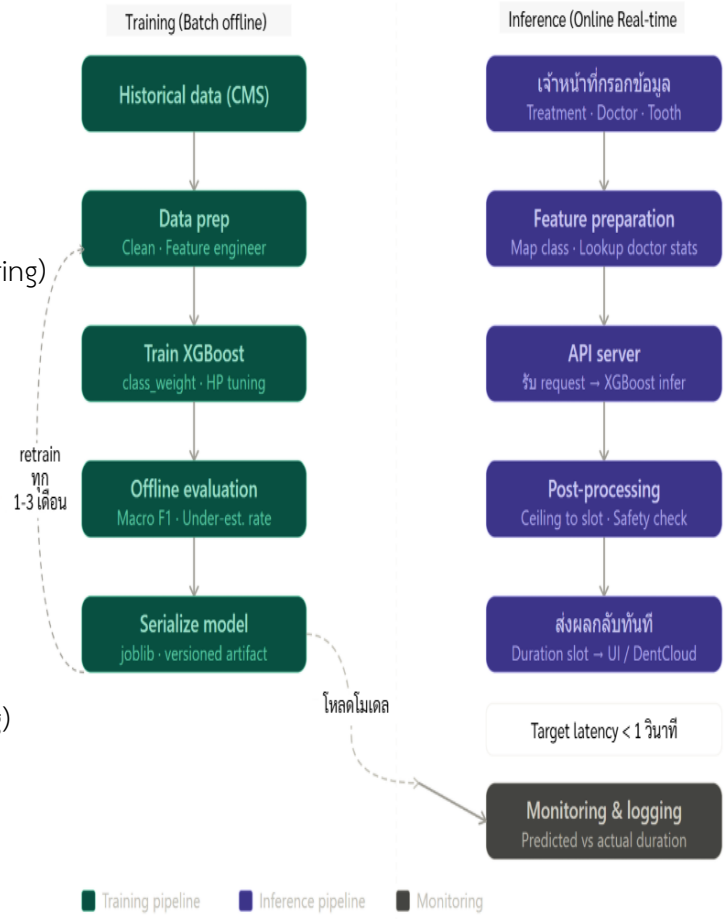
1. Project Overview

โครงการ DentTime มุ่งแก้ปัญหาการนัดหมายแบบกำหนดเวลาคงที่ในคลินิกทันตกรรม ซึ่งมักทำให้เกิด under-estimation และ over-estimation ของเวลารักษาในแต่ละเคส เมื่อจองเวลาสั้นเกินจริง ผู้ป่วยรายถัดไปจะรอนานขึ้น แต่เมื่อจองเวลานานเกินจริงจะเกิด gap time ที่ทำให้ทันตแพทย์และทรัพยากรถูกใช้ไม่เต็มประสิทธิภาพ
ดังนั้นระบบนี้จึงออกแบบมาเพื่อช่วยให้การจัดตารางนัดหมายสะท้อนความซับซ้อนของเคสจริงมากขึ้น
กลุ่มผู้ใช้งานหลักประกอบด้วย 3 กลุ่ม ได้แก่ เจ้าหน้าที่หน้ารับนัดที่ต้องตัดสินใจลงตารางนัดหมาย ทันตแพทย์ที่ต้องการตารางงานสมดุลขึ้น และผู้บริหารคลินิกที่ต้องการลด waiting time พร้อมเพิ่มประสิทธิภาพการใช้ทรัพยากรของหลายสาขา
ในเชิง Machine Learning ปัญหานี้ถูกนิยามเป็น supervised multi-class classification โดยระบบต้องจำแนกระยะเวลารักษาออกเป็น 7 ระดับ คือ 15, 30, 45, 60, 75, 90 และ 105 นาที เพื่อใช้เป็นช่วงเวลานัดหมายที่ปลอดภัยและใช้งานได้จริงในบริบทคลินิก
ชุดข้อมูลมาจาก Historical Appointment & Treatment Records ในระบบ Clinic Management System ครอบคลุมข้อมูลปี 2025 ประมาณ 1,000,000 records จากหลายคลินิกและหลายสาขา โดยใช้ฟิลด์หลัก เช่น appointment time, check-in time, treatment_record_time, receipt_time, tooth_no, treatments, notes และ doctor_license_no ซึ่งจะถูกทำ anonymization ด้วย SHA-256 ก่อนเข้าสู่ pipeline
แนวทางหลักของระบบคือสร้าง automated data cleaning pipeline สำหรับคัดเฉพาะคอลัมน์ที่จำเป็น จัดการ missing data, ทำ dictionary mapping และ fuzzy matching สำหรับข้อความหัตถการ, ตัด outlier และสร้าง feature เช่น Treatment Class, Tooth Count, Time of Day, Is First Case และ Doctor Profile Features จากนั้นเปรียบเทียบ Baseline กับ Random Forest และ XGBoost ภายใต้ data split และ preprocessing ชุดเดียวกัน โดยเลือก XGBoost เป็นโมเดลหลักสำหรับขั้นถัดไป และเก็บ Random Forest เป็น fallback model
ความคืบหน้าในรอบนี้ไม่ได้หยุดอยู่เพียงการออกแบบโมเดล แต่ขยายมาสู่มุมมอง ML System อย่างครบวงจร ได้แก่ สถาปัตยกรรม training/inference, วงจรชีวิตโมเดล, แนวทาง retrain, infrastructure ที่เหมาะกับขนาดโครงการ, แผน monitoring/data shift และเหตุผลของ prediction mode ที่จะใช้จริงในระบบนัดหมาย

2. MLS Architecture and ML Pipeline Design

ระบบ DentTime ถูกออกแบบเป็น Machine Learning System
ที่ประกอบด้วยองค์ประกอบหลักตั้งแต่การจัดการข้อมูล การฝึกโมเดล ไปจนถึงการใช้งานจริง
ดังแสดงในภาพรวมของระบบ

Clinic System (Raw Data)
↓
Data Storage (MySQL)
↓
Data Pipeline (Cleaning + Feature Engineering)
↓
Training Pipeline (XGBoost Model Training)
↓
Model Registry (Version Control)
↓
Prediction API (FastAPI)
↓
Web Application (Appointment Scheduling)
↓
Monitoring & Logging



ระบบ DentTime ถูกออกแบบเป็น ML System ที่มีการแยกส่วนระหว่าง **training pipeline** และ **inference pipeline** อย่างชัดเจน

- **Training Pipeline**
  ใช้ข้อมูลย้อนหลังจากระบบคลินิก มาผ่านขั้นตอน cleaning และ feature engineering ก่อนนำไปฝึกโมเดล XGBoost เพื่อสร้างโมเดลเวอร์ชันใหม่
- **Inference Pipeline**
  เมื่อมีการลงนัดใหม่ ระบบจะส่งข้อมูลผู้ป่วยไปยัง Prediction API เพื่อทำนายระยะเวลาการรักษาแบบ real-time และแสดงผลให้เจ้าหน้าที่ทันที

สรุป Pipeline

Pipeline	ขั้นตอนหลัก	วัตถุประสงค์
Training Pipeline	Data ⟶ Cleaning ⟶ Feature Engineering ⟶ Training ⟶ Evaluation	สร้างและเลือกโมเดลที่ดีที่สุด
Inference Pipeline	Input ⟶ Feature ⟶ Prediction ⟶ Scheduling ⟶ Logging	ทำนายและนำไปใช้จริง

ความแตกต่างระหว่าง Training และ Inference

มิติ	Training Pipeline	Inference Pipeline
วัตถุประสงค์	สร้างและประเมินโมเดลจากข้อมูลประวัติ	นำโมเดลมาทำนาย Duration สำหรับนัดหมายใหม่
Input	Historical records 1 ปี (batch ใหญ่)	on-demand ตามอาการ
Doctor Profile Stats	คำนวณจาก Training Set	ใช้ค่าที่คำนวณไว้แล้ว (Lookup)
Frequency	Periodic (ตาม Retrain Schedule)	on-demand ตามอาการ
Latency	ยอมรับได้หลายชั่วโมง	On-demand < 3 วินาที
Output	Trained Model + Metrics Report	Duration Slot per Appointment

2.2 Training และ Inference Design

2.2.1 Data Splitting

ข้อมูลถูกแบ่งโดยใช้วิธี chronological split ตามลำดับเวลา โดยแบ่งเป็น training, validation และ test set เพื่อสะท้อนการใช้งานจริงและป้องกัน data leakage นอกจากนี้ feature บางประเภท เช่น สถิติของทันตแพทย์ จะถูกคำนวณจาก training set เท่านั้น เพื่อหลีกเลี่ยงการใช้ข้อมูลอนาคต

Dataset	ช่วงเวลา	วัตถุประสงค์
Training Set	เดือนที่ 1–10	Train Model + คำนวณ Doctor Profile Stats
Validation Set	เดือนที่ 11 (ขยายจากเดือน 10 ถ้า Minority < 10%)	Hyperparameter Tuning + Model Selection
Test Set	เดือนที่ 12	Final Evaluation (ใช้ครั้งเดียว)

2.2.2 Evaluation

การประเมินผลใช้หลายตัวชี้วัดเพื่อสะท้อนทั้งด้านเทคนิคและธุรกิจ ได้แก่

- Macro F1-score เป็นตัวชี้วัดหลัก เพื่อให้ทุกคลาสมีความสำคัญเท่ากัน
- Under-estimation Rate เพื่อวัดความเสี่ยงจากการทำนายเวลาสั้นเกินจริง
- MAE (นาที) เพื่อวัดความคลาดเคลื่อนเชิงเวลา

2.2.3 Batch vs Online Prediction

ทางทีมเลือกใช้รูปแบบ **Online Prediction (On-demand/Synchronous)** แทนการประมวลผลแบบ Batch ด้วยเหตุผลดังต่อไปนี้:

- **Business Use Case:** ระบบ DentTime ทำหน้าที่เป็น Decision Support System (DSS) ให้กับเจ้าหน้าที่คลินิกในขณะที่กำลังรับจองคิว
  ทันทีที่เจ้าหน้าที่กรอกข้อมูลอาการหรือรายละเอียดเบื้องต้น ระบบจำเป็นต้องส่งคืนค่า Suggestion Time Window (ช่วงเวลาหัตถการที่เหมาะสม) กลับมาในทันที
  เพื่อประกอบการตัดสินใจลงตารางนัดหมาย
- **ความต้องการด้าน Latency:** คาดหวัง **End-to-End Latency** (รวมระยะเวลาของ Network I/O, Data Preprocessing, Model Inference และ Postprocessing) ให้อยู่ในระดับ p99 ไม่เกิน 3 วินาที เพื่อให้ User Experience (UX) ของแอปพลิเคชันมีความลื่นไหลและเสถียรสูงสุด
  เจ้าหน้าที่สามารถจัดตารางนัดหมายได้โดยไม่ต้องสะดุดหรือรอโหลดข้อมูลนาน

3. ML Lifecycle

หัวข้อนี้อธิบายภาพรวมวงจรชีวิตของโมเดล DentTime ตั้งแต่การเตรียมข้อมูลจนถึงการติดตามผลลัพธ์หลัง Deploy โดยแบ่งเป็นขั้นตอนที่ยังดำเนินการแบบ Manual, แนวคิดในการทำ Automation ในอนาคต และแนวทาง Retrain โมเดล

3.1 ภาพรวม ML Lifecycle ของโครงการ DentTime

วงจรชีวิตของโมเดล DentTime ประกอบด้วย 6 ขั้นตอนหลัก ดังนี้

#	ขั้นตอน	รายละเอียด	สถานะปัจจุบัน
1	Data Collection & Extraction	ดึงข้อมูลนัดหมายจาก CMS, ทำ Anonymization ด้วย SHA-256	Manual (SQL + Script) in Airflow
2	Data Cleaning & Feature Engineering	จัดการ Missing Data, Text Mapping, Outlier Removal, สร้าง Features	Semi-auto in Airflow
3	Model Training & Tuning	Train XGBoost, Tune Hyperparameters บน Validation Set	Manual (MLFLow)
4	Offline Evaluation	ประเมินบน Test Set ด้วย Macro F1, Under-estimation Rate, MAE	Manual (MLFLow)
5	Model Deployment	Export โมเดล, สร้าง REST API สำหรับ Batch Prediction	Manual (Flask/FastAPI)
6	Monitoring & Feedback	ตรวจสอบ Accuracy เทียบ Actual, ติดตาม Data Drift	ยังไม่ Implement

3.2 ขั้นตอนที่ยังทำแบบ Manual

ในขั้นตอนปัจจุบันกระบวนการส่วนใหญ่ยังดำเนินการแบบ Manual หรือ Semi-automated เนื่องจากโครงการอยู่ในระยะ Proof of Concept (PoC)

Data Extraction: ทีมดึงข้อมูลจาก Database ของ CMS ผ่าน SQL Query เขียนเฉพาะ แล้ว Export เป็น CSV จากนั้น Run Python Script เพื่อทำ Anonymization (SHA-256 Hashing) กับ License No ก่อนนำเข้า Pipeline ขั้นตอนนี้ต้องประสานงานกับทีม DBA เพื่อขอสิทธิ์เข้าถึงข้อมูล และต้อง Run ด้วยตนเองทุกครั้งที่ต้องการข้อมูลใหม่ โดยเขียนบน Airflow เพื่อให้ง่ายต่อการ Management

Data Cleaning & Feature Engineering: ใช้ Jupyter Notebook ที่รวมขั้นตอนทั้งหมดไว้ในไฟล์เดียว ตั้งแต่ Missing Data Handling, Fuzzy Match + Dictionary Mapping สำหรับ Treatment Class, Outlier Removal จนถึง Feature Creation อย่างไรก็ตาม Dictionary Mapping ต้องอัปเดตด้วยมือเมื่อพบรายการหัตถการใหม่ที่ยังไม่ตรงกับ Dictionary เดิม และ Doctor Profile Statistics ต้องคำนวณใหม่ทุกรอบ Retrain จาก Training Data เท่านั้น โดยเขียนไว้บน Airflow เพื่อให้ง่ายต่อการ Management

Model Training & Evaluation: การ Train โมเดลและ Tune Hyperparameters ทำผ่าน MLFLow โดยใช้ Grid Search / Random Search บน Validation Set ผลลัพธ์ (Metrics, Confusion Matrix, Feature Importance) บันทึกเป็น MLFlow Output การเปรียบเทียบระหว่าง Model Version ต้องตรวจสอบด้วยตนเอง

Model Deployment: โมเดลที่ผ่านการประเมินจะถูก Export เป็น Pickle หรือ Joblib แล้ว Deploy บน Flask/FastAPI Server ที่รับ Request จากระบบนัดหมายในลักษณะ On-demand Prediction การอัปเดตโมเดลใหม่ต้องทำ Manual Swap บน Server

3.3 แนวคิดในการ Automate ในอนาคต

แม้ในระยะ PoC จะยังไม่จำเป็นต้อง Implement ระบบ Automation เต็มรูปแบบ แต่ทีมได้วางแนวคิดไว้สำหรับการขยายผล ดังนี้

ขั้นตอน	สถานะปัจจุบัน	แนวคิด Automation
Data Extraction	Manual SQL Query	Scheduled ETL Pipeline (เช่น Airflow DAG) ดึงข้อมูลเดือนล่าสุดอัตโนมัติทุกสิ้นเดือน พร้อม Anonymization ในตัว
Data Validation	ตรวจสอบด้วยตาใน Notebook	ใช้ Great Expectations ตรวจจับ Schema Change, Missing Rate ผิดปกติ, Distribution Shift
Dictionary Update	อัปเดตด้วยมือ	ระบบ Flagging อัตโนมัติ สำหรับรายการที่ Fuzzy Match Score ต่ำกว่า Threshold แจ้งเตือนทีม Review
Model Training	Jupyter Notebook	แปลง Notebook เป็น Python Script + CLI พร้อม Config File สำหรับ Hyperparameter Grid
Evaluation & Registry	เปรียบเทียบด้วยตนเอง	ใช้ MLflow หรือ W&B สำหรับ Experiment Tracking และ Model Registry

Deployment	Manual File Swap	CI/CD Pipeline (GitHub Actions) ที่ Auto-deploy เมื่อโมเดลผ่านเกณฑ์ Evaluation
Monitoring	ยังไม่ Implement	Dashboard (Grafana) แสดง Accuracy, Drift, Under-estimation Rate แบบ Rolling Window โดยเก็บ Matric มาจาก Prometheus

3.4 แนวทาง Retrain โมเดล

3.4.1 เมื่อไหร่ควร Retrain

การ Retrain ควรพิจารณาจากหลายปัจจัยร่วมกัน ไม่ใช่เพียงระยะเวลาอย่างเดียว

- Time-based Trigger: กำหนด Retrain ทุก 3 เดือน (Quarterly) โดยใช้ข้อมูลย้อนหลัง 12 เดือนล่าสุดเป็น Rolling Training Window
- Performance-based Trigger: หาก Macro F1 บน Production Data ลดลงมากกว่า 5% จาก Baseline ที่เคยวัดได้ หรือ Under-estimation Rate เพิ่มขึ้นอย่างมีนัยสำคัญ
- Data Drift Trigger: หาก Distribution ของ Feature สำคัญเปลี่ยนแปลง เช่น มีหัตถการใหม่เข้ามา หรือสัดส่วนเคสยาวเปลี่ยน
- Event-based Trigger: เมื่อมีการเปลี่ยนแปลงเชิงระบบ เช่น เพิ่มสาขาใหม่, เปลี่ยน Workflow คลินิก, หรือมีทันตแพทย์ใหม่จำนวนมาก

3.4.2 กระบวนการ Retrain

กระบวนการ Retrain ออกแบบให้สอดคล้องกับ Chronological Split ที่ใช้ใน Training ครั้งแรก

#	ขั้นตอน	รายละเอียด
1	รวบรวมข้อมูลใหม่	ดึงข้อมูลจาก CMS ครอบคลุม 12 เดือนล่าสุด (Rolling Window) ทำ Anonymization และ Data Validation ตามขั้นตอนเดิม
2	อัปเดต Dictionary & Doctor Profile	ตรวจสอบหัตถการใหม่ที่ไม่อยู่ใน Dictionary, เพิ่ม Mapping ใหม่, คำนวณ Doctor Profile Statistics ใหม่จาก Training Data
3	Train โมเดลใหม่	ใช้ Hyperparameter Grid เดิมเป็น Starting Point, Fine-tune บน Validation Set ใหม่, ใช้ Early Stopping
4	เปรียบเทียบกับโมเดลเดิม	ประเมินบน Test Set ใหม่ เทียบ Macro F1, Under-estimation Rate, MAE กับโมเดลปัจจุบัน

5	ตัดสินใจ Deploy	หากดีกว่าหรือเทียบเท่าโมเดลเดิม ⟶ Deploy แทนที่ / หากแย่กว่า ⟶ ตรวจสอบสาเหตุก่อน
6	Archive โมเดลเดิม	เก็บไว้เป็น Fallback พร้อมบันทึก Metadata (Version, Training Period, Metrics)

3.4.3 Doctor Profile Update Strategy

Doctor Profile Features เป็น Aggregated Statistics ของทันตแพทย์แต่ละท่าน จึงต้องมีกลยุทธ์การอัปเดตชัดเจน ทันตแพทย์ที่มีข้อมูลเพียงพอจะคำนวณ Profile ใหม่จาก Training Window 12 เดือนล่าสุด สำหรับทันตแพทย์ใหม่ที่ไม่มีประวัติ จะใช้ Global Median เป็น Fallback ในระยะแรก แล้วค่อยอัปเดตเมื่อมีข้อมูลสะสมเพียงพอ (Cold-start Strategy) โดยกำหนดให้ต้องมีอย่างน้อย 30 เคสต่อหัตถการจึงจะใช้ค่า Doctor-specific ได้ หากน้อยกว่านี้ให้ใช้ Global Median แทน

3.4.4 Versioning & Rollback Plan

ทุก Model Version จะถูกบันทึกพร้อม Metadata ได้แก่ Training Period, Feature Set Version, Hyperparameters, Dictionary Version และ Evaluation Metrics ทีมจะเก็บโมเดลอย่างน้อย 2 Version ก่อนหน้าไว้เป็น Fallback และใช้ Naming Convention คือ denttime_xgb_v{MAJOR}.{MINOR}_{YYYYMM}.pkl เช่น denttime_xgb_v1.0_202501.pkl

4. Infrastructure และ Tooling

หัวข้อนี้อธิบายสภาพแวดล้อมและเครื่องมือที่ใช้ในโครงการ DentTime โดยเน้นความเหมาะสมกับขนาดโครงการในระยะ PoC ไม่จำเป็นต้องใช้เครื่องมือระดับ Production-grade ทุกตัว

4.1 การจัดเก็บข้อมูล (Data Storage)

ข้อมูลต้นทาง: ข้อมูลนัดหมายและการรักษาจริงจัดเก็บอยู่ใน Relational Database (MySQL/PostgreSQL) ของระบบ CMS ที่ให้บริการคลินิก ทีมดึงข้อมูลด้วย SQL Query เฉพาะ Columns ที่ต้องการ แล้ว Export เป็น CSV จากนั้นทำ SHA-256 Hashing ทันทีก่อนจัดเก็บในเครื่อง

ข้อมูลที่ผ่านการ Clean: จัดเก็บเป็น Apache Parquet เนื่องจากมีขนาดเล็กกว่า CSV อ่านเร็วกว่า และเก็บ Schema/Type ได้ชัดเจน

4.2 ทรัพยากรที่ใช้ Train โมเดล

เนื่องจาก XGBoost และ Random Forest เป็น Tree-based Models ที่ไม่ต้องการ GPU ทรัพยากรที่ใช้จึงอยู่ในระดับที่เข้าถึงได้ง่าย

รายการ	สเปค / รายละเอียด
สภาพแวดล้อมหลัก	Google Colab (Free/Pro) หรือ Local Machine
CPU	4–8 Cores เพียงพอสำหรับ XGBoost Training
RAM	16 GB ขึ้นไป (รองรับ Dataset ~1M records)
GPU	ไม่จำเป็น
Storage	~5–10 GB สำหรับ Raw Data, Processed Data, Model Artifacts
เวลา Training โดยประมาณ	RF ~5–15 นาที, XGBoost ~10–30 นาที (ขึ้นอยู่กับ Hyperparameter Grid)

4.3 การ Version Code

โครงสร้างโค้ดจัดการผ่าน Git Repository, ใช้ Git Flow แบบเรียบง่ายที่เหมาะกับทีมขนาดเล็ก โดย main เก็บโค้ดพร้อม Deploy, develop สำหรับ Feature ใหม่ และ experiment/* สำหรับทดลอง เช่น experiment/asymmetric-loss

4.4 การบันทึกผลการทดลอง

ในระยะ PoC ใช้วิธีบันทึกที่เรียบง่ายแต่เพียงพอ ได้แก่ MLFlow Outputs สำหรับ Metrics และ Confusion Matrix, Experiment Log เป็น CSV ที่บันทึก Experiment ID / Date / Model Type / Hyperparameters / ผลลัพธ์ และ Model Metadata เป็น JSON ที่คู่กับทุก Model Artifact

เมื่อโครงการเข้าสู่ระยะ Production วางแผนเปลี่ยนไปใช้ MLflow (ตัวเลือกหลัก เนื่องจากเป็น Open-source, Self-hosted ได้ ไม่มีค่าใช้จ่ายเพิ่ม) หรือ Weights & Biases (หากต้องการ Visualization และ Team Collaboration ที่ดีกว่า) สำหรับ Experiment Tracking และ Model Registry

4.5 Tech Stack Summary

หมวด	เครื่องมือ / Library	เวอร์ชัน
ภาษาโปรแกรม	Python	3.10+
Data Processing	Pandas, NumPy	pandas 2.x, numpy 1.x
Text Matching	RapidFuzz	ล่าสุด

ML Framework	scikit-learn, XGBoost	sklearn 1.x, xgboost 2.x
Notebook	Jupyter / Google Colab	-
Data Format	Parquet (PyArrow)	pyarrow 14+
Visualization	Matplotlib, Seaborn	ล่าสุด
Code Versioning	Git + GitHub	-
Model Serialization	Pickle / Joblib	Built-in
Demo Serving	Flask หรือ FastAPI	ล่าสุด
Experiment Log	CSV + JSON (PoC) ⟶ MLflow (อนาคต)	-

4.6 ข้อจำกัดและ Trade-offs

Infrastructure ปัจจุบันมีข้อจำกัดที่ต้องพิจารณาเมื่อขยายผล

ข้อจำกัด	ผลกระทบ	แนวทางแก้ไข (อนาคต)
Manual Data Pipeline	ใช้เวลาเตรียมข้อมูลทุกรอบ เสี่ยง Human Error	Airflow / Prefect + Great Expectations
ไม่มี Experiment Tracking	เปรียบเทียบข้าม Experiment ลำบาก	MLflow / W&B
Manual Deployment	Update โมเดลด้วยมือ ไม่มี Staging	CI/CD + Canary Deployment
ไม่มี Monitoring	ไม่ทราบเมื่อโมเดล Degrade	Grafana Dashboard + Alerting
Local/Colab Training	ขึ้นกับเครื่อง ไม่ Reproducible 100%	Docker Container / Cloud VM

โดยสรุป Infrastructure ปัจจุบันออกแบบให้เหมาะสมกับขนาดโครงการในระยะ PoC ใช้เครื่องมือที่เข้าถึงง่าย ค่าใช้จ่ายต่ำ และเพียงพอสำหรับพิสูจน์แนวคิด เมื่อพร้อมขยายผลสู่ Production สามารถยกระดับทีละขั้นตามแนวทางที่วางไว้ โดยเริ่มจาก Experiment Tracking และ Automated Data Valid

5. Data Shift และ Monitoring

5.1 ความเป็นไปได้ของ data distribution shift

เนื่องจากข้อมูลมีลักษณะเป็น Time-series และมาจากหลายคลินิก จึงมีความเสี่ยงต่อการเกิด Data Distribution Shift ได้ในหลายรูปแบบ ดังนี้

5.1.1 Covariate Shift (Data Drift):

- **พฤติกรรมการบันทึก:** การเปลี่ยนแปลงคำย่อหรือรูปแบบการกรอก Free Text ในฟิลด์ **treatments** และ **Notes** ซึ่งส่งผลต่อกระบวนการ Feature Extraction
- **ช่วงเวลาในการให้บริการ:** การเปลี่ยนช่วงเวลาที่มีการนัดหมาย (เช่น เปิดบริการเช้ามากขึ้น) ทำให้ distribution ของ Time-related features เปลี่ยน
- **ลักษณะเคสที่เข้ามา:** เช่น สัดส่วนเคสที่มี Tooth Count สูง/ต่ำ เปลี่ยนไป

5.1.2 Label Shift:

- **สัดส่วนหัตถการในแต่ละช่วงเวลา:** ทำให้ distribution ของ **Treatment Class** แตกต่างจากช่วงที่ใช้ฝึกโมเดล
- **นโยบายคลินิก:** เช่น โปรโมชั่นอุดฟันทำให้คลาส 30 นาทีสูงขึ้น หรือนโยบายรับเคสยากทำให้คลาส 90-105 นาทีเพิ่มขึ้นอย่างมีนัยสำคัญ

5.1.3 Concept Drift:

- **Doctor Profile:** ทันตแพทย์มีประสบการณ์เพิ่มขึ้น ทำให้ใช้เวลาน้อยลงในหัตถการเดิม ส่งผลให้ pattern ที่โมเดลเคยเรียนรู้เริ่มล้าสมัย (Model Staleness)

5.1.4 ML-specific Failures:

- **Training-Serving Skew:** ความไม่สอดคล้องของ Logic ของการ training และ serving เช่น วิธี preprocess ข้อความจาก **notes** ที่ใช้ Code คนละชุด
- **Label Noise & Feedback Loop:** ความคลาดเคลื่อนจากการบันทึกเวลา (เช่น check-out ล่าช้า) ทำให้ ground truth ไม่แม่นยำ และส่งผลให้ metric ที่ monitor ไม่สะท้อน performance จริง

5.2 metric ที่ควรติดตาม

5.2.1 Operational & System Metrics

- **Inference Latency:** ระยะเวลาที่ใช้ในการตอบสนองต่อคำขอ
- **Throughput (RPS):** จำนวนคำขอต่อวินาที
  เพื่อประเมินความสามารถในการรองรับการใช้งานจากหลายคลินิก
- **API Error Rate:** อัตราความผิดพลาดของระบบ
- **Availability / Uptime:** ความพร้อมใช้งานของระบบ เพื่อไม่ให้กระทบระบบนัดหมาย
- **CPU & Memory Utilization:** ใช้ตรวจสอบการใช้ทรัพยากร และรองรับการทำ Auto-scaling
- **Model Staleness:** ระยะเวลาตั้งแต่การเทรนครั้งล่าสุด เพื่อประเมินความจำเป็นในการ Retrain

5.2.2 Data & Prediction Metrics

- **Prediction Distribution (Output Drift):** ตรวจสอบสัดส่วนการทำนายในแต่ละคลาสเวลา (15-105 นาที) หากพบว่าคลาสใดคลาสหนึ่งมีความถี่สูงขึ้นอย่างผิดปกติ (เช่น คลาส 90 นาทีพุ่งสูงขึ้นกะทันหัน) จะถือเป็นสัญญาณแรกของ Drift
- **Feature & Prediction Stability:** ใช้ **Population Stability Index (PSI)**
  เพื่อวัดระดับการขยับเขยื้อนของการกระจายตัวข้อมูล โดยแบ่ง Time Scale Window ดังนี้:
  - **Rolling Window 7 วัน:** เปรียบเทียบข้อมูล Production รายสัปดาห์กับ Training Data เพื่อตรวจจับความผิดปกติระยะสั้น (เช่น ระบบดึงข้อมูลผิดพลาด หรือพฤติกรรมการจองที่เปลี่ยนไปตามช่วงวันหยุด)
  - **Cumulative Window รายเดือน:** ใช้สำหรับการวิเคราะห์แนวโน้มระยะยาว (Long-term Trend) เพื่อดูการขยับของพฤติกรรมคนไข้หรือนโยบายคลินิกที่ค่อยๆ เปลี่ยนไป (Concept Drift)
- **Multivariate Distribution Shift:** ใช้ **Maximum Mean Discrepancy (MMD)**
  เพื่อตรวจสอบความสัมพันธ์ระหว่างหลาย Features ที่เปลี่ยนไปพร้อมกัน

5.2.3 Performance & Business Metrics

- **Macro F1-score:** เนื่องจาก class imbalance
- **MAE:** วัดความคลาดเคลื่อนในเชิงเวลา

- **Under-estimation Rate:** วัดความเสี่ยงจากการทำนายเวลาน้อยเกินจริง ซึ่งกระทบต่อ waiting time
- **Schedule Adherence:**
  ประเมินความสอดคล้องระหว่างเวลานัดหมายกับเวลาที่เกิดขึ้นจริงในคลินิก โดยพิจารณาจาก:
  - **Average Gap Time** (เวลาว่างเฉลี่ยของทันตแพทย์)
  - **Average Overtime** (เวลาที่เกินจาก slot ที่จองไว้)
- **Early Warning Signals (Indirect Feedback)**: เนื่องจาก Accuracy metrics มักมี Latency สูง (ต้องรอ Ground Truth หลังหัตถการเสร็จหรือสรุปยอดรายเดือน)
  จึงต้องใช้ตัวชี้วัดทางอ้อมที่เร็วกว่า:
  - **User/Clinic Feedback**: การร้องเรียนจากทันตแพทย์เรื่องตารางนัดไม่ตรงจริง หรือคนไข้รอนานผิดปกติ
  - **System Logs**: ความถี่ที่เจ้าหน้าที่เคาน์เตอร์กด "แก้ไข" (Override) เวลาที่โมเดลแนะนำ

5.3 แนวทางเบื้องต้นเมื่อ performance ลดลง

5.3.1 Drift Verification

เมื่อระบบแจ้งเตือน ควรตรวจสอบตามลำดับความสำคัญและความเร็ว ดังนี้:

1. **Prediction Distribution:** ดูว่าโมเดลยังทำนายกระจายตัวปกติไหม โดยไม่ต้องรอ Label
2. **Feature Drift (PSI):** ตรวจสอบ Univariate drift ในฟีเจอร์หลัก (เช่นประเภทหัตถการ)
3. **Multivariate Drift (MMD):** ตรวจสอบความสัมพันธ์ระหว่างตัวแปร
4. **Performance Metrics:** ตรวจสอบ F1-Score/MAE เมื่อได้ Ground Truth ครบถ้วน

5.3.2 Model Retraining Strategy

1. Data Selection ตามประเภท Drift:

- **Covariate Shift:** ใช้ข้อมูลล่าสุด (Rolling Window 3-6 เดือน)
  เพื่อให้สอดคล้องกับปัจจุบัน
- **Label Shift:** ใช้ข้อมูลทั้งหมดแต่ทำ **Importance Weighting** หรือ ทำ **Cost-sensitive Tuning** โดยให้ Penalty ไม่เท่ากัน โดยตั้งค่าให้โทษของการทำนาย Under-estimation สูงกว่า Over-estimation

- **Concept Drift:** ปรับปรุง Feature Engineering (เช่น อัปเดต Doctor Profile) โดยใช้ recent data window (ช่วง 3-6 เดือน) และพิจารณา Retrain from scratch หากความสัมพันธ์เดิมใช้ไม่ได้แล้ว

2. Trigger Conditions

- **Time-based**: ทุก 2-4 สัปดาห์ (Scheduled)
- **Performance-based**: F1-score ลดลง > 5% จาก baseline
- **Drift-based**: PSI > 0.25 ใน feature สำคัญ
- **Business-based**: Under-estimation Rate เพิ่มขึ้นอย่างมีนัยสำคัญ

3. Rollback Plan:

- **Automated Rollback**: หาก Performance metrics ลดลงเกิน threshold ใน 24 ชั่วโมงแรก
- **Manual Rollback**: มี process ให้ทีมสามารถ rollback ได้ทันที
- **Fallback to Baseline**: ใช้ Fixed-duration Rule ชั่วคราวหากทั้ง current และ new model มีปัญหา

4. Retraining Approach:

- **Fine-tune model:** ใช้หาก Drift ไม่รุนแรงเพื่อรักษา Knowledge เดิม
- **Retrain from scratch:** ใช้หากเกิดการเปลี่ยนแปลงเชิงโครงสร้าง (เช่น เปิดสาขาใหม่ที่มีวัฒนธรรมองค์กรต่างกันสิ้นเชิง)

6. Deployment and Prediction Mode

ในการนำโมเดล Machine Learning ไปใช้งานจริง (Production) สำหรับระบบจัดการคิวคลินิกทันตกรรม (DentTime) ทางทีมได้ออกแบบสถาปัตยกรรมการให้บริการโมเดล (Model Serving Architecture) และกลยุทธ์การอัปเดตโมเดล ดังนี้

6.1 รูปแบบการให้บริการโมเดล (Prediction Mode)

ทางทีมเลือกใช้รูปแบบ **Online Prediction (On-demand/Synchronous)** แทนการประมวลผลแบบ Batch ด้วยเหตุผลดังต่อไปนี้:

- **Business Use Case:** ระบบ DentTime ทำหน้าที่เป็น Decision Support System (DSS) ให้กับเจ้าหน้าที่คลินิกในขณะที่กำลังรับจองคิว
  ทันทีที่เจ้าหน้าที่กรอกข้อมูลอาการหรือรายละเอียดเบื้องต้น ระบบจำเป็นต้องส่งคืนค่า Suggestion Time Window (ช่วงเวลาหัตถการที่เหมาะสม) กลับมาในทันที
  เพื่อประกอบการตัดสินใจลงตารางนัดหมาย
- **ความต้องการด้าน Latency:** คาดหวัง **End-to-End Latency** (รวมระยะเวลาของ Network I/O, Data Preprocessing, Model Inference และ Postprocessing) ให้อยู่ในระดับ p99 ไม่เกิน 3 วินาที เพื่อให้ User Experience (UX) ของแอปพลิเคชันมีความลื่นไหลและเสถียรสูงสุด
  เจ้าหน้าที่สามารถจัดตารางนัดหมายได้โดยไม่ต้องสะดุดหรือรอโหลดข้อมูลนาน

6.2 ข้อจำกัดหากจำนวนผู้ใช้เพิ่มขึ้น และแผนการรองรับ (Deployment & Scaling)

ในอนาคต หากมีการขยายการใช้งานไปยังคลินิกหลายสาขา ปริมาณ Concurrent Users และ Request ที่เข้ามาพร้อมกันจะเพิ่มสูงขึ้น เพื่อป้องกันปัญหาระบบหน่วงหรือล่ม ทางทีมได้ออกแบบสถาปัตยกรรม (System Design) บน **Kubernetes (GKE Autopilot)** เพื่อรองรับการขยายตัว (Scalability) โดยมี Pipeline ดังนี้:

- **Model Optimization:** นำโมเดลที่ดีที่สุดจากการทดลอง (เช่น XGBoost) มา Export ให้อยู่ในรูปแบบ **ONNX format** เพื่อเพิ่มประสิทธิภาพความเร็วในการทำ Inference และลดการใช้ทรัพยากร
- **Model Storage:** จัดโครงสร้างไฟล์ให้อยู่ในรูปแบบ **Triton Model Repository** และนำไปจัดเก็บไว้อย่างเป็นระบบบน Cloud Storage (เช่น GCS Bucket)
- **Model Serving:** ใช้ **KServe** ร่วมกับ Triton Inference Server ในการดึงโมเดลจาก Bucket มาให้บริการในรูปแบบ InferenceService API
- **Auto-scaling:** เมื่อมีปริมาณ Request เพิ่มขึ้น Kubernetes จะทำงานร่วมกับ KServe ในการทำ **Auto-scaling** (เพิ่มจำนวน Pod อัตโนมัติ) เพื่อกระจายโหลดการทำงาน

ทำให้ระบบสามารถรักษาระดับ Latency ให้เสถียรอยู่ที่ไม่เกิน 1 วินาทีได้ตามเป้าหมาย

6.3 กลยุทธ์การอัปเดตโมเดล (Deployment Strategy)

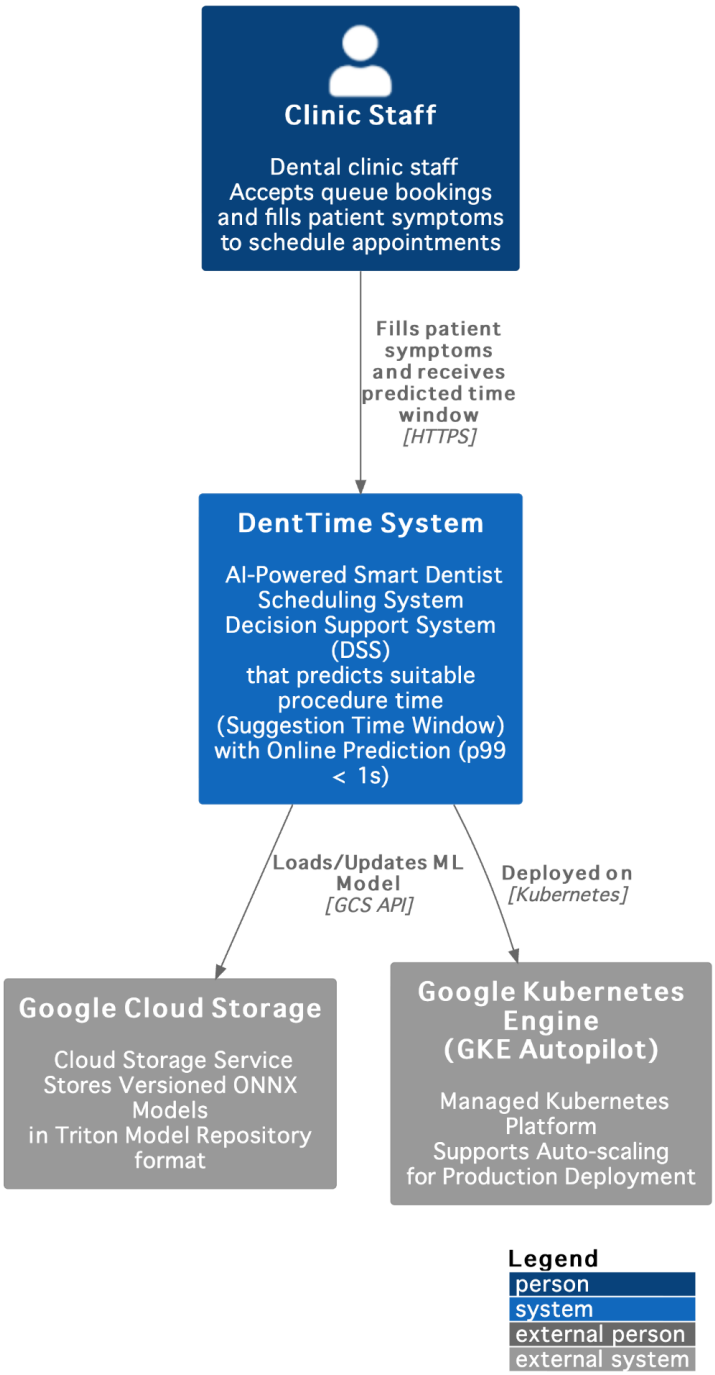
การนำโมเดลเวอร์ชันใหม่ (เช่น โมเดลที่ Retrain ด้วยข้อมูลของเดือนล่าสุด) ขึ้นสู่ Production มีความเสี่ยงทางธุรกิจ (Business Risk) หากโมเดลใหม่ทำงานคลาดเคลื่อนและทายเวลาผิดพลาด ทางทีมจึงกำหนดกลยุทธ์เพื่อลดความเสี่ยงดังนี้:

1. **Shadow Deployment:** ในระยะแรก จะรันโมเดลเวอร์ชันใหม่เป็น Background (Shadow) คู่ขนานไปกับโมเดลเดิม โดยให้ระบบจำลองการรับ Request จริงมาทำ Inference แต่จะยังไม่ส่งผลลัพธ์นี้กลับไปให้ User เพื่อประเมินผลการทำงานของโมเดลใหม่เทียบกับโมเดลเดิมอย่างเงียบๆ
2. **Canary Release:** หากโมเดลใหม่ใน Shadow Mode ผ่านเกณฑ์ประเมินที่ตั้งไว้ จะขยับไปใช้เทคนิค Canary Release โดยแบ่ง Traffic ของผู้ใช้จริงเพียง 10% ให้เข้ามาใช้งานโมเดลใหม่ หากระบบทำงานได้อย่างมีเสถียรภาพและไม่มีข้อผิดพลาด จึงจะค่อยๆ Rollout จนครบ 100%

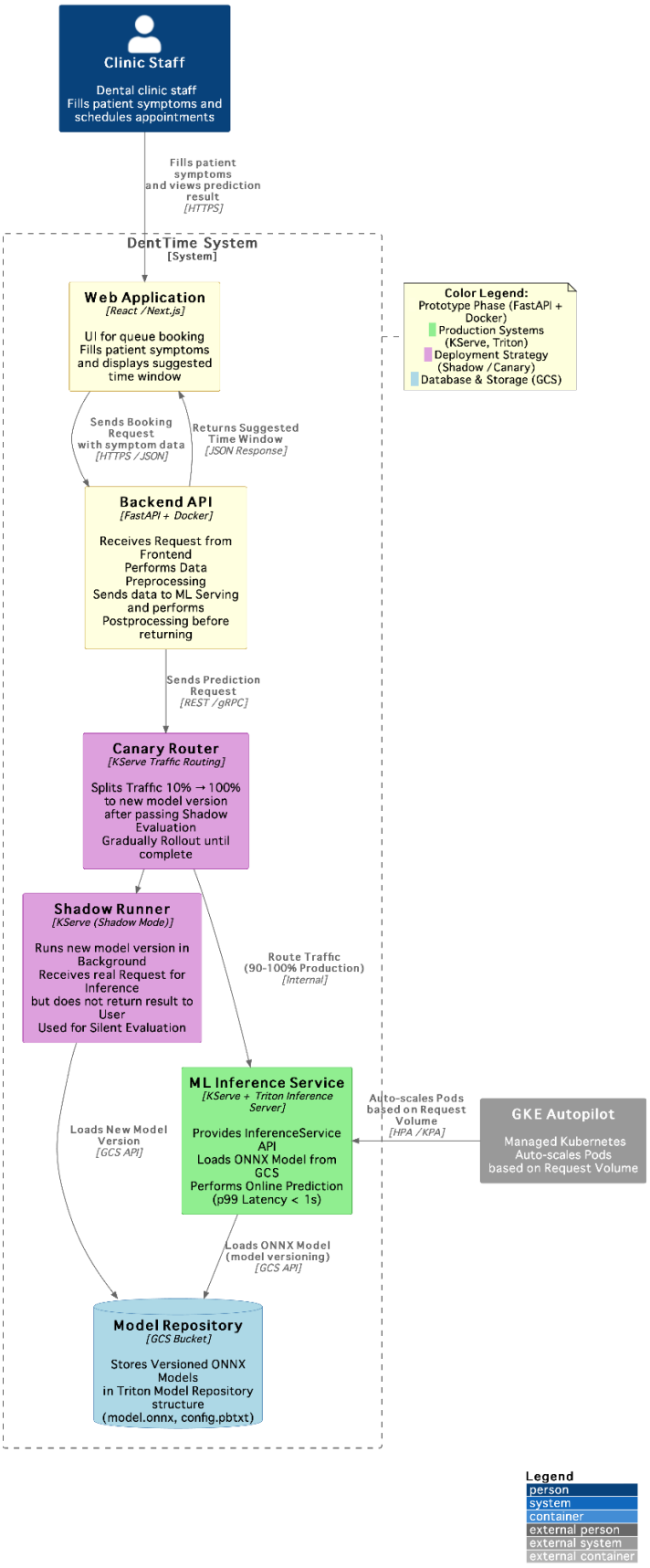
หมายเหตุ (Current Implementation Phase):
สำหรับเฟสการพัฒนา Prototype ในปัจจุบัน เพื่อความรวดเร็วในการทดสอบ (Proof of Concept) ทางทีมจะเริ่มพัฒนาระบบให้บริการโมเดลเบื้องต้นผ่าน **FastAPI และ Docker** เป็นหลักก่อน เพื่อให้สามารถเชื่อมต่อกับแอปพลิเคชันส่วนหน้าได้ และจะขยายผลไปสู่สถาปัตยกรรม K8S ในเฟสถัดไป

C4 Model

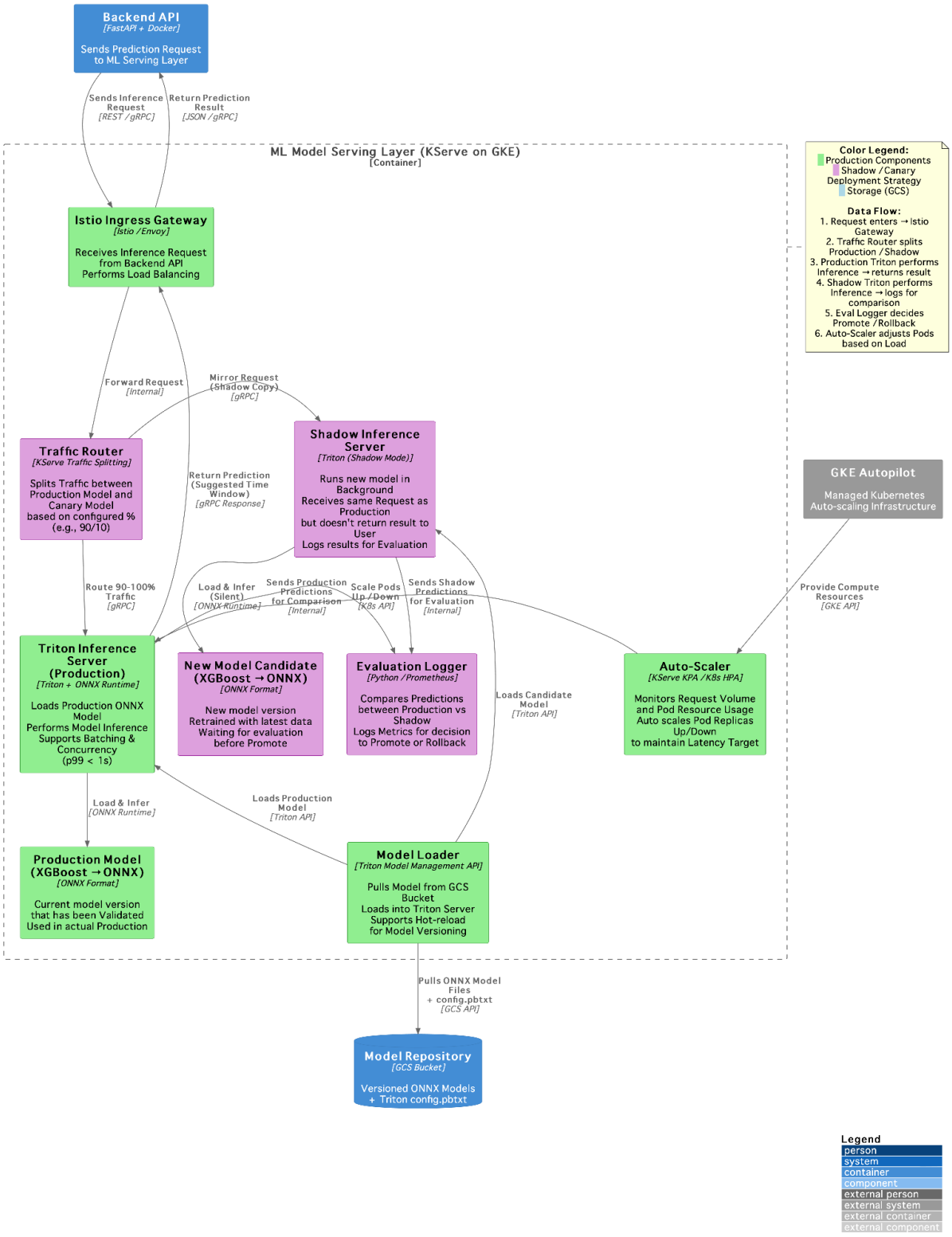
[System Context Diagram] DentTime - AI-Powered Smart Dentist Scheduling System



[Container Diagram] DentTime - AI-Powered Smart Dentist Scheduling System



[Component Diagram] DentTime - ML Model Serving Subsystem (Zoom into KServe + Triton)



7. การปรับปรุงจาก Progress 1

ตารางต่อไปนี้สรุปว่ารายงานฉบับนี้ปรับปรุงจาก Progress 1 อย่างไร
โดยอ้างอิงจากข้อเสนอแนะของอาจารย์และสิ่งที่ถูกเติมให้ครบในกรอบ Progress 2.

ข้อเสนอแนะ	การปรับปรุง	หลักฐานในรายงานฉบับนี้
เพิ่มผลเชิงตัวเลขให้ชัดขึ้น	ระบุกรอบ evaluation ชัดขึ้น ได้แก่ Macro F1, MAE, Under-estimation Rate และ Confusion Matrix เพื่อรองรับการเติมผลตัวเลขจริงในรอบถัดไป	Section 2.2.2, Section 4.4, Section 5.2
คิดเรื่อง deployment architecture เพิ่ม	เพิ่มภาพรวมสถาปัตยกรรมตั้งแต่ data storage, training pipeline, model registry และ prediction API จนถึง monitoring พร้อมแยก training กับ inference ให้ชัดเจน	Section 2, System Overview Diagram, Section 6
จะ predict แบบ batch หรือ real-time	เพิ่มการอธิบาย prediction mode ทั้งมุม batch/on-demand ใน pipeline design และเหตุผลด้าน latency ในส่วน deployment	Section 2.2.3, Section 6.1
handle doctor ใหม่อย่างไร	เพิ่ม cold-start strategy โดยใช้ Global Median สำหรับแพทย์ใหม่ก่อนมีข้อมูลเพียงพอ	Section 3.4.3
monitoring plan ถ้า workflow เปลี่ยน	เพิ่ม drift 4 แบบ, metric ที่ต้องติดตาม และแนวทางตอบสนองเมื่อ	Section 5.1–5.3

	performance ลดลง	
ยกระดับจาก model discussion ไปสู่ ML lifecycle	เพิ่ม ML lifecycle, automation roadmap, retrain trigger, versioning, rollback และ infrastructure ระดับ PoC	Section 3, Section 4, Section 6.2–6.3
ทดสอบ scenario-based simulation	ฉบับนี้ยังไม่เพิ่ม simulation เชิงตัวเลข แต่เพิ่ม business metrics เช่น schedule adherence, gap time และ overtime เพื่อวางฐานในรอบถัดไป	Section 5.2.3